



# Lab3 Verilog FIR

---

R12943168 陳瀚坪

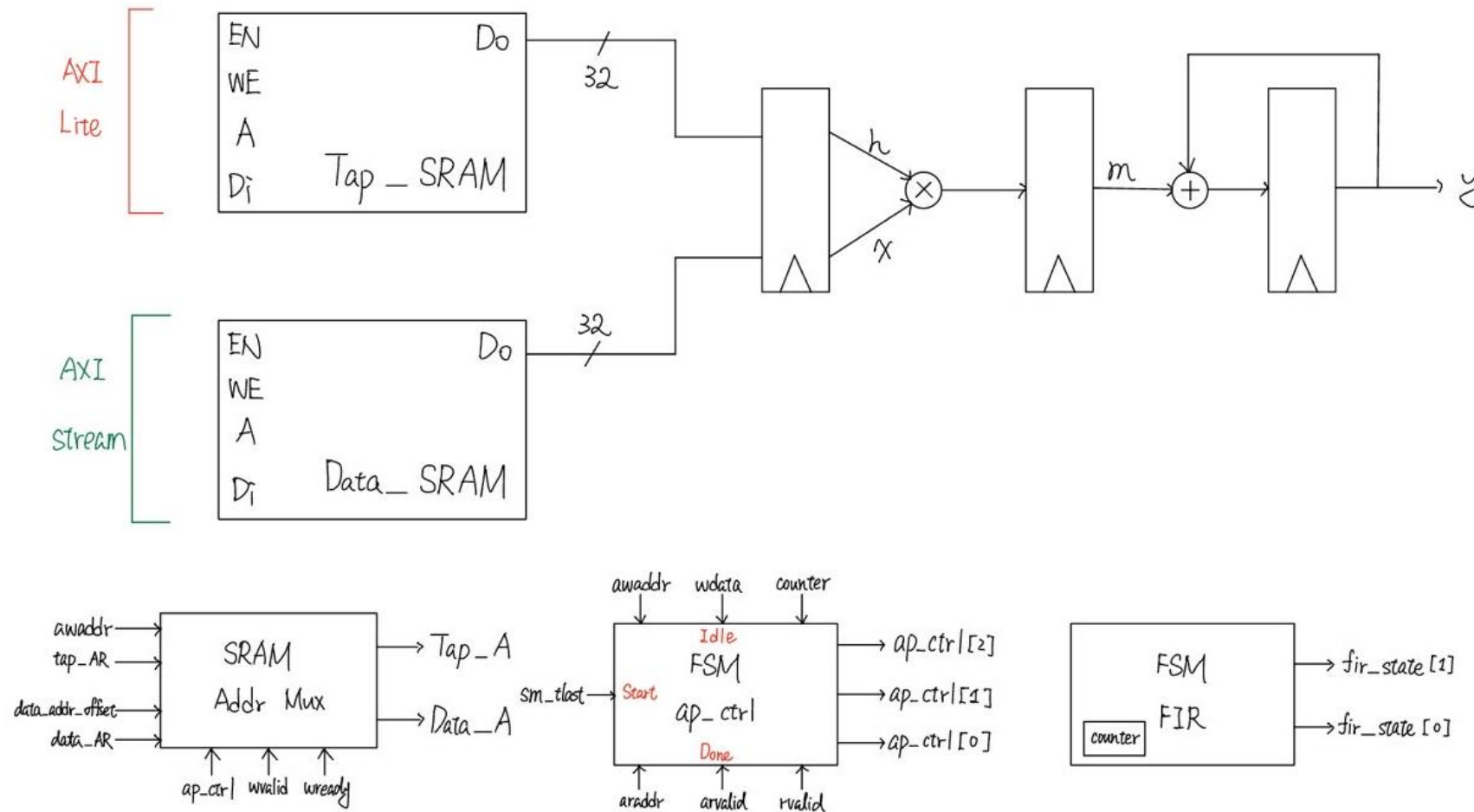
# Outline

---

- Block Diagram
- Waveform
- Tap/Data RAM access sequence & its control
- Axilite access timing
- Axistream X, Y access timing
- Special cases handling
- Testbench
- Code Review Questions

# Block Diagram

- Datapath



# Block Diagram

- FSM (ap\_ctrl)

```

113 // ----- Configuration Register control -----
114 assign ap_ctrl[2] = (ap_state == AP_IDLE)?1:0;
115 assign ap_ctrl[1] = (ap_state == AP_DONE)?1:0;
116 assign ap_ctrl[0] = (ap_state == AP_IDLE && awaddr == 12'd0 && tlast_cnt != data_length)?1:0;
117
118 // Jiin: explain the state machine below
119 always@(*) begin
120     case (ap_state)
121         // Jiin: awaddr, and wdata need to qualify with awvalid, wvalid otherwise they are invalid value
122         AP_IDLE : next_ap_state = (awaddr == 12'd0 && wdata[0] && tlast_cnt != data_length) ? AP_START : AP_IDLE;
123         AP_START: next_ap_state = sm_tlast ? AP_DONE : AP_START;
124         AP_DONE : next_ap_state = (araddr == 12'd0 && arvalid && rvalid) ? AP_IDLE : AP_DONE;
125         default : next_ap_state = AP_IDLE;
126     endcase
127 end
128
129 always@(posedge axis_clk or negedge axis_rst_n) begin
130     if (!axis_rst_n)
131         ap_state <= AP_IDLE;
132     else
133         ap_state <= next_ap_state;
134 end

```

# Block Diagram

- FSM (filter)

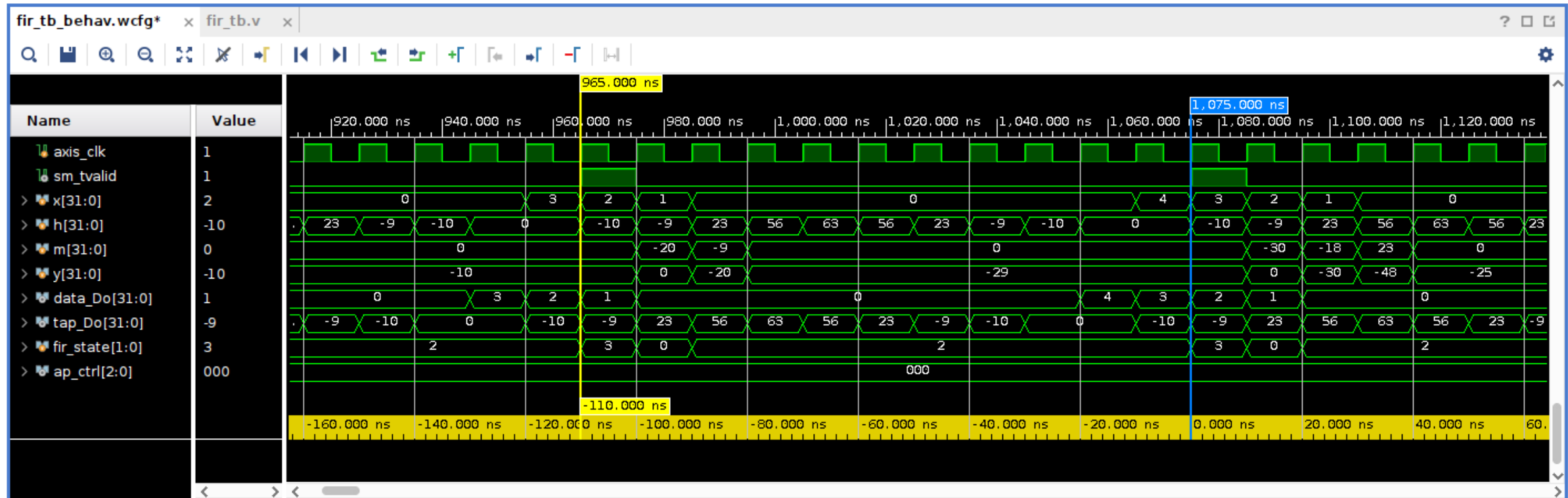
```

188 //----- AXI-Stream output Y -----
189 always@(posedge axis_clk or negedge axis_rst_n) begin
190     if (!axis_rst_n)
191         out_cnt <= 5'd0;
192     else if (fir_state == FIR_IDLE)
193         out_cnt <= (tlast_cnt == 0)? 10'd13 : 10'd8;           // Jiin: explain it
194     else if (fir_state == FIR_START || fir_state == FIR_PROC)
195         out_cnt <= out_cnt - 1'b1;
196     else
197         out_cnt <= out_cnt;
198 end
199 always@(*) begin
200     case (fir_state)
201     FIR_IDLE : begin
202         if(ap_ctrl[2] == 1'b0 && tlast_cnt == 0)
203             next_fir_state = FIR_START;
204         else if(ap_ctrl == 1'b0 && tlast_cnt != 0)
205             next_fir_state = FIR_PROC;
206         else
207             next_fir_state = FIR_IDLE;
208         end
209     FIR_START: next_fir_state = (out_cnt == 0)? FIR_DONE : FIR_START;
210     FIR_PROC : next_fir_state = (out_cnt == 0)? FIR_DONE : FIR_PROC;
211     FIR_DONE : next_fir_state = FIR_IDLE;           // Jiin: it shall wait for y is transferred, otherwise y is reset to 0
212     default  : next_fir_state = FIR_IDLE;
213     endcase
214 end

```

# Waveform

- Waveform (11T / per Y)
- ap\_ctrl
- fir\_state

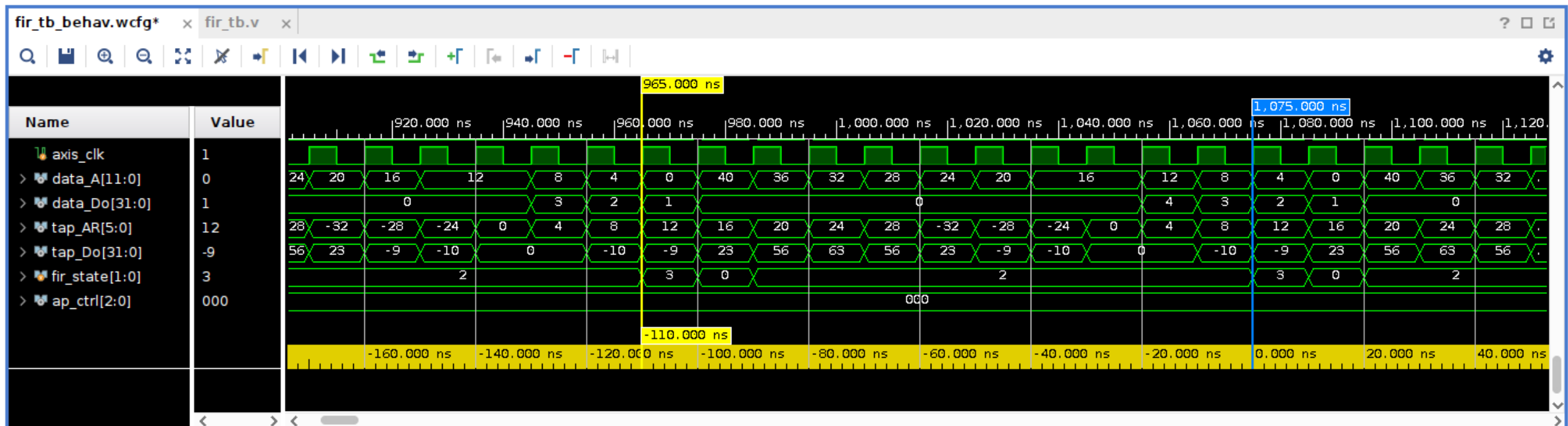


# Tap/Data RAM access sequence & its control

```

145 //----- tap / Data RAM signals ----- // 0x20-FF: tap parameter
146 assign awaddr_revise = (awaddr < 12'h80)? awaddr - 12'h20 : awaddr;
147 assign tap_EN = 1;
148 assign tap_WE = ((wvalid == 1) && (awaddr[7:0] != 0))? 4'b1111 : 4'b0000;
149 assign tap_A = (ap_ctrl[2] == 1 && (wvalid == 1 || wready == 1))? {6'd0,awaddr_revise[5:0]} : tap_AR[5:0];
150 assign tap_Di = wdata;
151
152 assign data_EN = 1;
153 assign data_WE = (ss_tready || init_done == 0)? 4'b1111 : 4'b0000; // if ss_tlast not asserted, still can write
154 assign data_A = (ap_ctrl[2] == 1 && init_done == 1)? 4*data_addr_offset : data_AR; // data initialize before ap_start
155 assign data_Di = (ap_ctrl[2] == 1 && init_done == 0)? 0 : ss_tdata; // 44 => 11-tap data

```



# Axilite access timing

```

92 //----- AXI-Lite -----
93 always@(posedge axis_clk, negedge axis_rst_n) begin
94     if(!axis_rst_n) begin
95         r_arready    <= 0;    // address read  ready
96         r_awready    <= 0;    // address write ready
97         r_rvalid     <= 0;    // data      read  valid
98         r_wready     <= 0;    // data      write ready
99     end
100     else begin
101         r_arready    <= (arvalid)?1 : 0; // address read  ready
102         r_rvalid     <= (arvalid | rvalid & ~rready)?1 : 0; // data      read  valid
103         r_awready    <= (awvalid && wvalid)?1 : 0;    // address write ready
104         r_wready     <= (awvalid && wvalid)?1 : 0;    // data      write ready
105     end
106 end
107 assign arready = r_arready;
108 assign awready = r_awready;
109 assign rvalid  = r_rvalid;
110 assign wready  = r_wready;
111 assign rdata   = (araddr[7:0] == 8'd0)? ap_ctrl : tap_Do; // read ap state or Tap SRAM data // Jiin: faulty, data length register ??

```



# Special cases handling

---



- Special Case
  - X input too fast / too slow
  - Y output too fast / too slow
- To Do List
  - Add Input data register
  - Add Output data register

# Testbench

```
182     initial begin
183         error = 0; status_error = 0;
184         sm_tready = 1;
185         wait (sm_tvalid);
186         for(k=0;k < data_length;k=k+1) begin
187             sm(golden_list[k],k);
188         end
189         config_read_check(12'h00, 32'h02, 32'h0000_0002); // check ap_done = 1 (0x00 [bit 1])
190         config_read_check(12'h00, 32'h04, 32'h0000_0004); // check ap_idle = 1 (0x00 [bit 2])
191         if (error == 0 & error_coef == 0) begin
192             $display("-----");
193             $display("-----Congratulations! Pass-----");
194         end
195         else begin
196             $display("-----Simulation Failed-----");
197         end
198         $finish;
199     end

228     reg error_coef;
229     initial begin
230         error_coef = 0;
231         $display("----Start the coefficient input(AXI-lite)----");
232         config_write(12'h10, data_length);
233         for(k=0; k< Tape_Num; k=k+1) begin
234             config_write(12'h20+4*k, coef[k]);
235         end
236         awvalid <= 0; wvalid <= 0;
237         // read-back and check
238         $display(" Check Coefficient ...");
239         for(k=0; k < Tape_Num; k=k+1) begin
240             config_read_check(12'h20+4*k, coef[k], 32'hffffffff);
241         end
242         arvalid <= 0;
243         $display(" Tape programming done ...");
244         $display(" Start FIR");
245         @(posedge axis_clk) config_write(12'h00, 32'h0000_0001); // ap_start = 1
246         $display("----End the coefficient input(AXI-lite)----");
247     end
```

# Code Review Questions

- FIR

```
224 //----- FIR -----
225 always @(posedge axis_clk or negedge axis_rst_n) begin
226     if (!axis_rst_n) begin
227         h <= 32'd0;
228         x <= 32'd0;
229         m <= 32'd0;
230         y <= 32'd0;
231     end
232     else if (ap_ctrl[2] == 1) begin
233         h <= 32'd0;
234         x <= 32'd0;
235         m <= 32'd0;
236         y <= 32'd0;
237     end
238     else begin
239         h <= tap_Do;
240         x <= data_Do;
241         m <= h * x;
242         y <= (fir_state == FIR_DONE)?0:y + m;
243     end
244 end
```

// Jiin: FIR\_DONE is only valid for 1T (see FSM above)

# Code Review Questions

## • Address

```

246 //----- Address -----
247 assign araddr_revise = (araddr < 12'h80)? araddr - 12'h20 : araddr; // Jiin: faulty logic
248 assign tap_AR = (ap_ctrl[2] == 1'b0)? 4 * data_addr_offset : {6'd0,araddr_revise[5:0]};
249
250 always@(*) begin
251     if (ap_state != AP_IDLE && data_addr_offset == 10)
252         x_cnt_tmp = (x_cnt != 10) ? x_cnt + 1 : 0;
253     else
254         x_cnt_tmp = x_cnt;
255 end
256
257 always @(posedge axis_clk or negedge axis_rst_n) begin
258     if (!axis_rst_n)
259         x_cnt <= 4'd0;
260     else
261         x_cnt <= x_cnt_tmp;
262 end
263
264 assign data_AR = (data_addr_offset <= x_cnt)? 4 * (x_cnt - data_addr_offset) : 4 * (11 + x_cnt - data_addr_offset); // x[t-i]

```

## • AXI-Stream

```

266 //----- AXI-Stream -----
267 // tvalid is driven by master / tready is driven by tslave
268 // Stream-in X
269
270 // Jiin: do we consider that Y is not transferred ...
271 assign ss_tready = (ap_ctrl[2] == 0 && init_done == 1 && data_addr_offset == 4'd0)? 1'b1 : 1'b0;
272 // Stream-out Y
273 assign sm_tvalid = (fir_state == FIR_DONE)? 1'b1 : 1'b0; // Jiin: sm_tvalid should wait for sm_tready
274 assign sm_tdata = y;
275 assign sm_tlast = (fir_state == FIR_IDLE && tlast_cnt == data_length)? 1'b1 : 1'b0;

```